

Image Processing Assignment

ME461 - Group: Little Daisies

Abdullah Naci Bodur Buğra Çınaroğlu Ogan Altuğ Okutan

Fall 2025-2026



Contents

- 1 Problem Definition
- 2 Methodology
- 3 Feature Extraction
- 4 Implementation Details
- 5 Conclusion



The assignment consists of two main tasks focused on Image Processing:

- **Task 1: Synthetic Data Generation**

- Create a GUI to generate random scenes with 10-15 objects.
- Objects must vary in **Shape, Size, and Color**.
- Implement adjustable noise parameters (0-1) for Color and Shape.

- **Task 2: Automated Solving**

- Process the generated image to identify objects.
- Assign a unique 3-digit ID (**XYZ**) to each object.
- Display the ID on the object center.



The XYZ Identification System

Each object is classified by a 3-digit code based on its properties:

X: Object Shape ID

- 1 = Star
- 2 = Rectangle
- 3 = Triangle

Y: Size ID

- 1 = Large
- 2 = Medium
- 3 = Small

Z: Color ID

- 1 = Red
- 2 = Purple
- 3 = Cyan



Approach Overview

Our solution follows the standard Image Processing pipeline:

- ① **Acquisition:** Synthetic generation via Python/OpenCV.
- ② **Pre-processing:** Grayscale conversion and Thresholding.
- ③ **Feature Extraction:** Computing Area, Compactness, and Mean Color.
- ④ **Classification:** Mapping features to IDs.
- ⑤ **Visualization:** Labeling the original image.



Pre-processing: Grayscale & Thresholding

- **Grayscale Conversion:** We convert the input RGB image to Grayscale to simplify data from 3 channels to 1.
- **Binary Thresholding:** Since our background is white and objects are colored, we use *Inverse Binary Thresholding* to create a mask where objects are white (1) and background is black (0).

Why?

This separates the foreground (objects) from the background, allowing for contour detection and analysis.



Feature Extraction: Contour Detection

We utilize **Contour Detection** to identify distinct objects in the binary mask.

- This is equivalent to finding **Connected Components**.
- It allows us to isolate each object and compute properties individually.
- We filter out extremely small contours to handle noise.



Identifying Size (Y ID)

Property Used: Area (A)

- The area is calculated as the summation of pixels within the object boundary:

$$A = \sum_{i=1}^n \sum_{j=1}^m B[i,j]$$

- **Selection Logic:** Since we generate objects with specific base radii, we define Area thresholds scaled by the resolution:
 - $A > T_{Large} \rightarrow \text{Large (1)}$
 - $A > T_{Medium} \rightarrow \text{Medium (2)}$
 - Else $\rightarrow \text{Small (3)}$



Property Used: Compactness (C)

- Compactness is derived from Area (A) and Perimeter (P):

$$C = \frac{P^2}{A}$$

- **Why Compactness?** It is a dimensionless quantity invariant to translation, rotation, and scaling. This is crucial as our objects are randomly rotated and scaled.



Shape Classification Logic

We distinguish shapes based on their theoretical Compactness values:

- **Circle:** Theoretical value ≈ 12.57

$$C < 15$$

- **Rectangle:** Closest to a square/circle in terms of perimeter efficiency.

$$15 < C < 19$$

- **Triangle:** Higher perimeter for the same area compared to a square.

$$19 < C < 26$$

- **Star:** High perimeter due to points/valleys relative to area.

$$26 < C$$



Identifying Color (Z ID)

Property Used: Mean Intensity & Euclidean Distance

- We apply the object mask to the original color image to isolate pixels.
- We calculate the **Mean BGR** vector for the object.
- We compute the **Euclidean Distance** between the mean color and our known database colors (Red, Purple, Cyan):

$$d_{Euclidean} = \sqrt{(R_1 - R_2)^2 + (G_1 - G_2)^2 + (B_1 - B_2)^2}$$

- The object is assigned the ID of the closest matching color.



Property Used: Position (Centroid)

- To label the object with "XYZ", we need its center position.
- We use **Image Moments** to calculate the Centroid $(\bar{x}, \bar{y})$:

$$\bar{x} = \frac{M_{10}}{M_{00}}, \quad \bar{y} = \frac{M_{01}}{M_{00}}$$

- This ensures the text is centered regardless of the object's position in the image.



Handling Noise

The assignment required robustness against noise.

- **Color Noise (Salt&Pepper):** If high color noise is detected/applied, we utilize a **Median Blur** filter to reduce noise while preserving edges before thresholding.
- **Shape Noise (Vertex Jitter):** Our use of **Area** and **Compactness** is naturally robust. While jitter increases perimeter (affecting Compactness), our thresholds (< 19 vs > 35) provide a large safety margin to distinguish shapes even with distortion.



We implemented a GUI using `tkinter` to meet the "Generate GUI" requirement.

- **Dynamic Preview:** Users can resize the preview window for high-res images (2K/4K).
- **Ground Truth:** Ability to toggle between "Noisy" and "Clean" images to verify solver accuracy.
- **Configuration:** Dropdowns to map specific Shapes and Colors to IDs 1, 2, and 3.



- We successfully developed a system to generate and solve synthetic scenes.
- **Key Methods Selected:**
 - **Thresholding:** For robust segmentation.
 - **Area:** For size detection
 - **Compactness:** For rotation/scale invariant shape detection.
 - **Euclidean Distance:** For accurate color matching.
- The solution handles scaling, random positioning, and noise effectively.



Questions?

